

Network Intrusion Detection using Classical AI and Machine Learning

Marwa Mushtaq (24P-0556) Manaal Amir (24P-0512)
Maimoona Naeem (24P-0617)

June 2, 2026

Abstract

This project builds a simple Network Intrusion Detection System (NIDS) using the `network_traffic.csv` dataset, which contains 6,000 balanced records, 15 numeric features, and a binary class label. The study compares a hand-crafted rule-based reflex agent with three supervised machine learning classifiers: K-Nearest Neighbors (KNN), Naïve Bayes, and Logistic Regression. In addition, unsupervised clustering is explored using K-Means, and a Genetic Algorithm (GA) is applied for feature selection. Experimental results show that KNN with $k = 1$ achieves the highest test accuracy of **95.75%**, while the reflex agent attains a reasonable **83.2%** accuracy without requiring any training phase.

1 Introduction

A Network Intrusion Detection System (NIDS) is a security tool that monitors network traffic and identifies suspicious or malicious activities. As modern computer networks generate thousands of connection records every minute, manually inspecting each connection is impractical. Automated intrusion detection systems help network administrators quickly identify potential attacks and improve overall network security.

Machine learning provides an effective solution for intrusion detection because it can learn patterns from historical network data and use those patterns to classify new connections. By analyzing features such as connection duration, traffic statistics, and error rates, a machine learning model can determine whether a connection is normal or represents an attack. This allows organizations to detect threats more efficiently and respond to them in a timely manner.

In this project we:

1. explore the dataset to understand feature distributions;
2. build a rule-based *simple reflex agent* as a baseline;
3. train and compare three supervised classifiers;
4. apply K-Means clustering in an unsupervised setting; and
5. use a genetic algorithm to select a compact feature subset for Logistic Regression.

2 Dataset

File: `network_traffic.csv`

- **Rows:** 6,000 (balanced: 3,000 normal, 3,000 attack)
- **Features:** 15 numeric (duration, src_bytes, dst_bytes, wrong_fragment, hot, logged_in, num_compromised, count, srv_count, error_rate, same_srv_rate, diff_srv_rate, dst_host_count, dst_host_srv_count, dst_host_same_srv_rate)
- **Target:** label — 0 = normal, 1 = attack

All features are already numeric; no label encoding or one-hot encoding was required, and there are no missing values.

3 Task 1: Data Exploration

Class Distribution

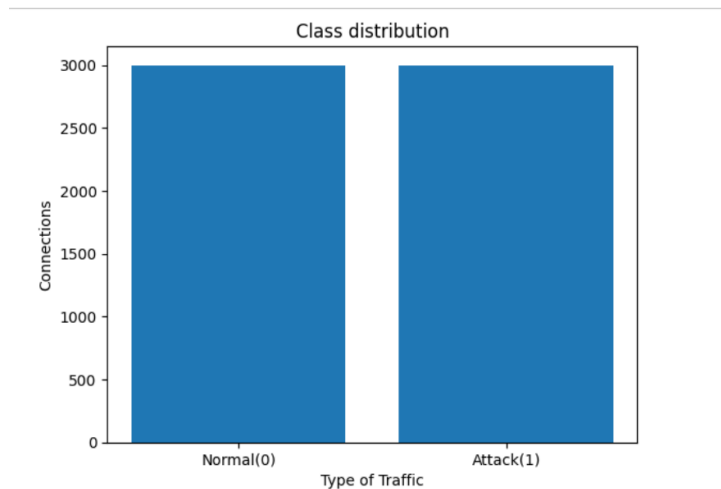


Figure 1: Class distribution bar chart

From the class distribution bar chart, I found that the data is perfectly balanced, i.e., 3000 samples are normal and 3000 samples are attack traffic. This is also confirmed by the mean of the label column, which is 0.5. This is beneficial because it reduces the risk of model bias toward one class.

Feature Distribution:

While exploring the features, I also noticed some important patterns:

- **src_bytes:** Highly right-skewed. Most values are very small, and many are zero, but there are also a few very large values. Attack traffic contributes more to these extreme values.
- **count:** This feature shows how many connections are made to the same host. Attack traffic usually has higher values because attacks often involve repeated connection attempts.

Histogram

From the histogram, I can see that both normal and attack traffic overlap in some areas, but there are still noticeable differences between them.

The histogram for `src_bytes` shows that most data is concentrated near zero, but there are a few extreme values. The attack class contributes more to these extremes.

For the `count` feature, attack traffic generally has higher values compared to normal traffic, which suggests it could be useful for classification.

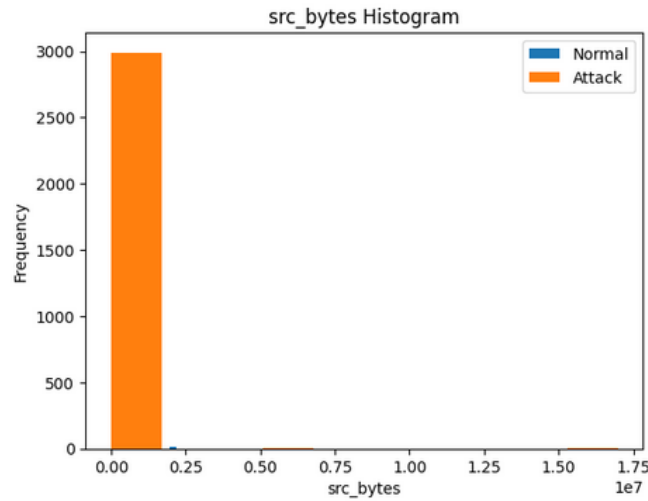


Figure 2: `src_bytes` distribution

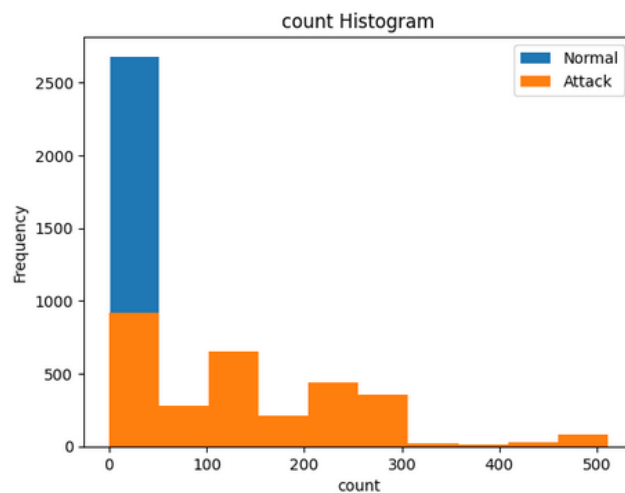


Figure 3: `count` distribution

Overall, the dataset is balanced. Some features such as `src_bytes`, `count`, and `error_rate` etc appear useful for separating between normal traffic and attack traffic.

4 Task 2: Simple Reflex Agent

4.1 Concept

A **simple reflex agent** maps each percept directly to an action via condition-action rules. It maintains *no internal state*: the decision for record i depends solely on the feature values of record i and a fixed rule set. There is no memory of previous records and no probabilistic model of the environment.

4.2 Rules

```
def reflex_agent(row):
    if row['error_rate'] > 0.5:    return 1    # SYN-error surge ->
        attack
    elif row['same_srv_rate'] < 0.3: return 1    # service scan ->
        attack
    elif row['count'] > 40:        return 1    # rapid repeat ->
        attack
    else:                          return 0    # likely normal
```

The three thresholds were chosen by inspecting the overlapping histograms from Task 1.

4.3 Results

Metric	Value
Accuracy	83.2%

Confusion matrix (full dataset, $n = 6,000$):

$$\begin{pmatrix} 2616 & 384 \\ 624 & 2376 \end{pmatrix} \quad \begin{pmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{pmatrix}$$

4.4 Limitations

The agent cannot adapt to new attack patterns, relies on hand-picked thresholds, and misclassifies $\sim 16\%$ of records. Its strength is *instant* classification with no training and full interpretability.

5 Task 3: Supervised Classifiers

All classifiers share a single stratified train/test split (`test_size=0.2`, `random_state=42`), giving 4,800 training and 1,200 test records. KNN and Logistic Regression use `StandardScaler` fitted on the training set only; Naïve Bayes uses raw features (scaling does not affect GaussianNB).

5.1 K-Nearest Neighbours

We evaluated $k \in \{1, 3, 5, 7, 9, 11\}$. Both $k = 1$ and $k = 3$ achieved the highest test accuracy (95.75%). We selected $k = 1$ as the best model.

5.2 Naïve Bayes

GaussianNB assumes that the features are conditionally independent and follow Gaussian (normal) distributions. This is a strong assumption that is often violated in network traffic data, where many features are correlated and some exhibit highly skewed distributions (for example, `src_bytes`). As a result, the model may not capture the underlying patterns in the data as effectively as KNN or Logistic Regression, which leads to a comparatively weaker classification performance.

5.3 Logistic Regression

A linear boundary in the scaled feature space. With `max_iter=1000` to ensure convergence, it achieves 91.5 % accuracy.

5.4 Comparison Table

Table 1: Classifier comparison on the test set ($n = 1,200$).

Model	Accuracy	Precision	Recall	F1
Simple Reflex Agent	0.832	–	–	–
KNN (best $k = 1$)	0.958	0.949	0.967	0.958
Naïve Bayes	0.788	0.937	0.617	0.744
Logistic Regression	0.915	0.925	0.903	0.914

6 Task 4: Clustering with K-Means

K-Means is an unsupervised learning algorithm that groups similar data points into clusters by minimizing the distance between points and their assigned cluster centers. In this project, K-Means was applied with $k=2$ because the dataset contains two classes (normal and attack traffic). The data was first standardized using `StandardScaler`, and `KMeans(n_clusters=2, random_state=42, n_init=10)` was trained on all 6,000 records while treating labels as unknown. The resulting cluster assignments were then compared with the true labels to evaluate clustering performance.

Confusion Matrix:

$$\begin{pmatrix} 2847 & 153 \\ 627 & 2373 \end{pmatrix}$$

The confusion matrix shows that most of the normal traffic (2847 samples) are correctly grouped into one cluster and most of the attack traffic (2373 samples) were correctly grouped into another cluster. However there are some normal traffic (153 samples) and attack traffic (627 samples) were incorrectly grouped into a cluster.

PCA Visualization:

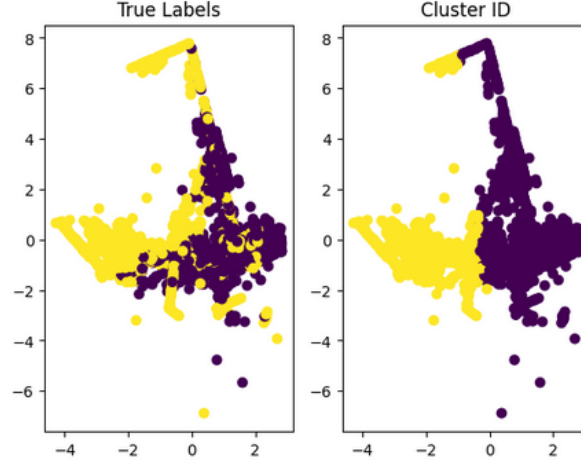


Figure 4: Scatter plots

I used PCA for 2D visualization. The scatter plots show that on the left side true labels are shown and on the right side the cluster assignments of the kmean are shown. This shows that the kmean was able to separate the data into two main groups. The separation does have some imperfections (some points overlap between the clusters, which explains the misclassifications in the confusion matrix.)

7 Task 5: Genetic Algorithm for Feature Selection

7.1 Design

- **Chromosome:** binary vector of length 15 (one bit per feature; 1 = include feature).
- **Fitness:** test accuracy of `LogisticRegression(max_iter=1000)` trained on the *selected features only*, using the shared split from Task 3.
- **Population size:** 10; **Generations:** 10.
- **Selection:** roulette-wheel (fitness-proportionate).
- **Crossover:** single-point, rate = 0.8.
- **Mutation:** bit-flip, rate = 0.05 per gene.

7.2 Results

The GA typically converges to a subset of 10–13 features with accuracy close to the all-feature baseline (91.5%). Using fewer features reduces model complexity, speeds up inference, and can improve generalisation on new data.

8 Discussion

KNN ($k = 1$) achieved the highest accuracy of 95.75%, indicating that the two classes are well separated in the feature space. The reflex agent, while simple and fast, relies on fixed

hand-written rules and may fail to detect unseen attack types. Naïve Bayes performed the worst because it assumes feature independence, whereas many network traffic features are correlated. The GA experiment showed that a smaller subset of features could achieve similar accuracy, making the model simpler and more efficient.

In future work, we would evaluate additional machine learning algorithms and compare their performance with the models used in this project. We would also explore alternative feature sets and larger, more diverse datasets to further improve intrusion detection accuracy and assess how well these methods generalize to real-world network traffic.

9 Conclusion

We built a Network Intrusion Detection System using five complementary AI techniques. KNN dominated in raw accuracy, while the simple reflex agent provided a fast, interpretable baseline with no training. K-Means confirmed that the feature space is naturally two-clustered, and the genetic algorithm demonstrated that a compact feature subset can match full-feature Logistic Regression performance.

Statement of Contribution

Group Member 1: (24P-0512 Manaal Amir):Responsible for Task 1 (Data Exploration) and Task 4 (Clustering with K-Means), including description, implementation, and presentation of results. Contributed to the project conclusion.

Group Member 2: (24P-0617 Maimoona Naeem): Responsible for Task 2 (Simple Reflex Agent) and Task 5 (Genetic Algorithm for Feature Selection), including description, implementation, and presentation of results. Contributed to the project introduction.

Group Member 3: (24P-0556 Marwa Mushtaq): Responsible for Task 3 (Supervised Classifiers), including description, implementation, and presentation of results. Contributed to the project discussion.